

Part 1:

See AutoCAD File called Hardware Configuration with TIA Portal Type Configuration. It mimics how TIA Portal schematic should be but since I have no internet currently and have problems I had to resort to AutoCAD. The logic is nevertheless the same.

Part 2:

```
SENSOR_A := "3.3 B5 Proximity";
```

```
SENSOR_B := "3.3 B4 Proximity";
```

```
PNEUMATIC_VALVE := "3.3Y1";
```

```
// If sensor A doesn't notice an object, then it is true or logic 0 else it is logic 1.
```

```
// If sensor B doesn't notice the cylinder or when the cylinder is far away it is false or logic 0 else
```

```
// it is logic 1.
```

```
// Note that sensor A and sensor B have reverse logic. This is done on purpose to simplify the
```

```
// the program and make it clearer.
```

```
// When sensor 3.3B5 detects products, valve 3.3Y1 shall be activated so that the cylinder moves
```

```
// forward and blocks the packages.
```

```
// When sensor 3.3B5 no longer detects products, the valve shall switch so that the cylinder retracts
```

```
// and releases the packages.
```

```
IF SENSOR_A THEN
```

```
    PNEUMATIC_VALVE := FALSE;
```

```
ELSE
```

```
    PNEUMATIC_VALVE := TRUE;
```

```
ENDIF
```

```
// Sensor 3.3B4 shall be used to confirm that the cylinder has fully reached its rear position.
```

```
IF PNEUMATIC_VALVE = FALSE AND SENSOR_A = TRUE THEN
```

```
    // Small NOP, wait. This follows from previous program. One simply uses a small wait time
```

```
    IF SENSOR_A = FALSE THEN // and checks the status of sensor A. From there one can easily
```

```
        SENSOR_B := TRUE; // determine when the cylinder has reached its rear position.
```

```
    ENDIF
```

```
    IF SENSOR_A = TRUE THEN
```

```
        SENSOR_B := FALSE;
```

```
    ENDIF
```

```
ENDIF
```

Part 3:

For Part 3 I will simply implement this by writing in SCL. The analog input for valve activation time in HMI and similar objects will be linked to tags for a global data block called HMI_Data. Suppose the HMI Input is VALVE_ACTIVATION. This will be defined under HMI_DATA. One assumes HMI_DATA.VALVE_ACTIVATION will be set by user.

```
// If timer starts at 0 and is activated and sensor A doesn't sense any object and sensor B
// senses the rear of the cylinder. Then this is the initial step.
IF (HMI_DATA.VALVE_ACTIVATION = TRUE AND SENSOR_A = TRUE) AND
(HMI_DATA.VALVE_ACTIVATION = TRUE AND SENSOR_B = TRUE) THEN

    PNEUMATIC_VALVE := TRUE;

ENDIF

// If timer starts at 0 and is activated and sensor A senses an object and sensor B
// doesn't sense the rear of the cylinder. Then this is an undefined step and the timer is reset back to
// zero. At this undefined step the packages are blocked during the reset after a brief NOP the
// pneumatic valve is reactivated again.
IF (HMI_DATA.VALVE_ACTIVATION = TRUE AND SENSOR_A = FALSE) AND
(HMI_DATA.VALVE_ACTIVATION = TRUE AND SENSOR_B = FALSE) THEN

    HMI_DATA.VALVE_ACTIVATION := FALSE;

    // Small NOP delay

    HMI_DATA.VALVE_ACTIVATION := TRUE;

    PNEUMATIC_VALVE := TRUE;

ENDIF
```

```

// If timer ends and elapses and is deactivated and sensor A senses an object and sensor B
// doesn't yet sense the rear of the cylinder. Then one first activates the pneumatic valve then
// after a healthy waiting period the pneumatic valve becomes deactivated. Then one waits until
// sensor B senses the end of the cylinder to reset until it becomes true. Then one waits until sensor
// A detects an object for the pneumatic valve to be activated once again.
IF (HMI_DATA.VALVE_ACTIVATION = FALSE AND SENSOR_A = FALSE) AND
(HMI_DATA.VALVE_ACTIVATION = FALSE AND SENSOR_B = FALSE) THEN

    PNEUMATIC_VALVE := TRUE;

    // Small NOP delay

    PNEUMATIC_VALVE := FALSE;

    IF SENSOR_B = TRUE THEN

        // NOP delay

        IF SENSOR_A = FALSE THEN

            PNEUMATIC_VALVE := TRUE;

        ENDIF

    ENDIF

ENDIF

```

```

// If timer elapses and is deactivated and sensor A doesn't sense any object and sensor B
// doesn't sense the rear of the cylinder. Then this is an undefined step and so one waits until
// both sensor A notices an object and sensor B notices the rear of the cylinder. The timer
// is reactivated to zero and the pneumatic valve is immediately activated. And the
// process restarts in cycles from there on.
IF (HMI_DATA.VALVE_ACTIVATION = FALSE AND SENSOR_A = TRUE) AND
(HMI_DATA.VALVE_ACTIVATION = FALSE AND SENSOR_B = FALSE) THEN

    IF SENSOR_B = FALSE OR SENSOR_A = TRUE THEN

        // NOP, wait until sensor A becomes FALSE and sensor B becomes TRUE

    ENDIF

    IF SENSOR_B = TRUE AND SENSOR_A = FALSE THEN

        HMI_DATA.VALVE_ACTIVATION := TRUE;
        PNEUMATIC_VALVE := TRUE;

    ENDIF

ENDIF

```

Part 4:

See AutoCAD file.

Part 1 Update: Internet started working but don't have time since I used the web version of TIA Portal I had to use Siemens S7-1500 CPU 1511C-1 PN 6ES7 511-1C400-0AB0 version. The actual programming steps above would have been the same. So I didn't redo it again.



PNEUMATIC_VA...	Bool	%Q4.0	Default tag table
	Bool	%Q4.1	
	Bool	%Q4.2	
	Bool	%Q4.3	

Bonus:

Finally for the alarm system implementation. One simply defines variable called ALARM. The logic would be an addition to the previous program one defined earlier.

```
MAX_TIME = "X s";
```

```
COUNTER = 0;
```

```
ALARM = "red light";
```

```
IF PNEUMATIC_VALVE := FALSE AND SENSOR_A := TRUE THEN
```

```
    // Small NOP, wait
```

```
    IF SENSOR_A := FALSE THEN
```

```
        SENSOR_B := TRUE;
```

```
        COUNTER--;
```

```
    ENDIF
```

```
    IF SENSOR_A := TRUE THEN
```

```
        SENSOR_B := FALSE;
```

```
    ENDIF
```

```
    COUNTER++;
```

```
    IF COUNTER > MAX_TIME THEN
```

```
        ALARM;
```

```
    ENDIF
```

```
ENDIF
```